Day 142: GANs vs VAEs vs Diffusion: Real, Honest Quality/Speed Tradeoffs

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 3 | Complete study guide

Watch: Generative Models 101: VAEs, GANs, Autoregressive & Diffusion Models Explained (CorestratAI) | Denoising Diffusion Implicit Models (DDIM) Explained (ExplainingAI)

Script: `day142_generative_tradeoffs.py` · every number in this guide is copied by program from `day142_run_output.txt`, the real output of that script.

Three families of generative model have now been built from scratch: Week 19's VAE, Week 20's GAN and Day 141's diffusion model. Today all three are trained on the same 60,000 MNIST digits, scored by the same scorer and timed on the same machine. The diffusion model, sampled from an averaged copy of its weights, scored best (FID 27.778, against the VAE's 38.520 and the GAN's 52.781), but it trained longest (1244s against 100s and 125s) and took 97.06s to draw 2,000 digits where the others took 0.02s. DDIM cut sampling to 2.42s with 20 steps, at a cost: FID 61.201. And the scorer missed something the eye sees at once: the VAE's samples are blurred, which a direct count of grey pixels shows and FID does not.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

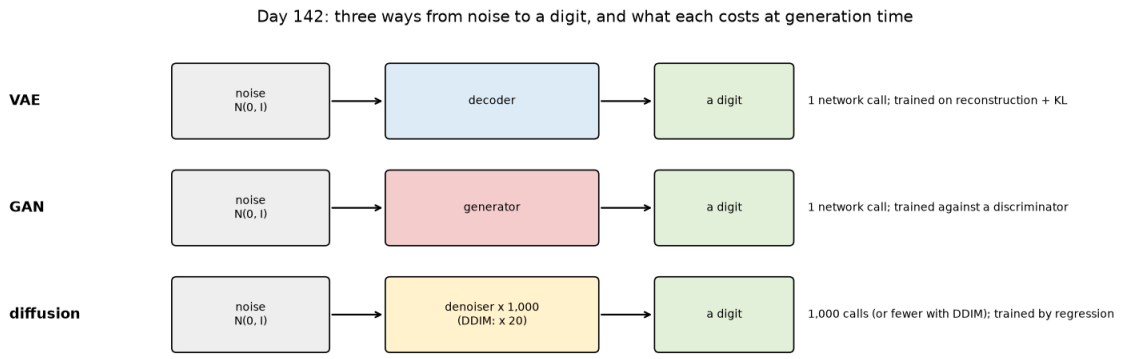
A **generative model** learns from examples and then makes new ones. Three families have been built in this course: the **VAE** (Days 128–130) compresses each image to a few numbers and learns to decode random numbers into images; the **GAN** (Days 131–139) trains a generator against a critic; the **diffusion model** (Days 139–141) learns to undo noise and generates by removing it step by step.

All three are judged on **quality** (do the samples look real, and do they cover every kind of digit?) and on **cost** (how long training takes, and how long one sample takes). Today puts real numbers on both, for all three, in one place.

REAL-WORLD SCENARIO

A team choosing an image generator for a product: a phone keyboard that suggests stickers must answer in milliseconds; a design tool that renders one poster can wait a minute for a better image. The same three families make different trades between those two needs, and the right choice depends on which one the product cannot give up.

SYSTEM DIAGRAM



The three families on one scorer: one network call per sample for the VAE and GAN; up to 1,000 for diffusion, which DDIM cuts.

Today's part for the week's build

`day142_exercise.py` is today's parts (the averaged copy of the weights, DDIM, the x_0 guesser wrapper, the grey share and the weight count), checked by hand: one averaging step, DDIM with η 1 and 1,000 steps reproducing Day 140's sampler, the grey share of a made-up image, and a briefly trained model sampled in 20 steps.

ACTUAL OUTPUT

```
one averaging step, decay 0.9: weights 0 -> 0.10 (0.9 x 0 + 0.1 x 1), biases 1 -> 0.90 (0.9 x 1 +
    0.1 x 0)
DDIM, eta 1, 1,000 steps against Day 140's sampler: largest difference 2.9e-14
an image with 100 ink pixels and 50 at 0.5: grey share 0.0638 (50 / 784 = 0.0638)
a model trained 1,000 steps (1,083,152 numbers), 100 samples in 20 DDIM steps: ink 0.185, grey
    share 0.1846
```

Data: Day 126's 70,000 MNIST digits (training images 0–59,999; test images 60,000 on, always taken in random order); pixels in [0, 1] for the VAE and GAN, scaled to [–1, 1] for diffusion. The VAE is Day 129's, sampled with Day 130's prior sampler; the GAN step is Day 134's; the diffusion model is Day 141's; the scorer is Day 138's. All are imported unchanged. Every time is measured on this machine with 4 threads; the run was repeated after the first one was slowed by the laptop going into standby (section 6).

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: DDIM's step, checked against Day 140's sampler

DEFINITION — start here, no prior knowledge assumed

Day 140's sampler walks back one step at a time, 1,000 times, adding fresh noise at each step. **DDIM** (Song et al., 2021) takes the same trained model and jumps: from step t it guesses x_0, works out the noise that guess implies, and puts both back together at an earlier step s, perhaps 50 steps earlier. A setting η controls fresh noise: $\eta = 0$ adds none after the start, $\eta = 1$ adds as much as Day 140's sampler.

REAL-WORLD SCENARIO

A hiker with a good map who can skip the trail markers: instead of walking to each marker in turn, they read the map at one marker and walk straight to one far ahead. Fewer stops, but each straight line cuts corners the trail did not.

MATHEMATICS

$$x_s = \sqrt{A_s}\, x'_0 + \sqrt{(1 - A_s - \sigma^2)}\, \varepsilon' + \sigma\, z, \quad z \sim N(0, I) \text{ fresh}$$

$$\varepsilon' = (x_t - \sqrt{A_t}\, x'_0) / \sqrt{(1 - A_t)}$$

$$\sigma^2 = \eta^2\, (1 - A_s) / (1 - A_t) \cdot (1 - A_t / A_s)$$

A_t = ABAR[t]; primes mark the network's guesses; $s < t$ is the step jumped to

IMPLEMENTATION

```
def ddim_sample(x0_fn, n, rng, steps, eta=0.0, d=784):
    ts = np.linspace(len(ABAR) - 1, 0, steps + 1).round().astype(int)
    x = rng.standard_normal((n, d))
    for t, s in zip(ts[:-1], ts[1:]):
        x0 = x0_fn(x, t); e = (x - np.sqrt(ABAR[t]) * x0) / np.sqrt(1 - ABAR[t])
        sig = eta * np.sqrt((1 - ABAR[s]) / (1 - ABAR[t]) * (1 - ABAR[t] / ABAR[s]))
        x = np.sqrt(ABAR[s]) * x0 + np.sqrt(np.maximum(1 - ABAR[s] - sig ** 2, 0)) * e
        if s > 0:
            x = x + sig * rng.standard_normal(x.shape)
    return x
```

ACTUAL OUTPUT

```
DDIM's fresh noise at a jump from t to s: sigma^2 = eta^2 (1 - abar_s) / (1 - abar_t) (1 - abar_t / abar_s);
with eta = 1 and one step at a time (s = t - 1) it should equal Day 140's beta-tilde_t
     t  DDIM sigma^2, eta 1  beta-tilde_t
     2         0.0000545319  0.0000545319
    10         0.0002381820  0.0002381820
   100         0.0020345963  0.0020345963
   500         0.0100313554  0.0100313554
  1000         0.0199999835  0.0199999835
Day 140's sampler against DDIM with eta 1 and all 1,000 steps, same guesser, same random numbers: largest difference
    3.58e-14
DDIM with eta 0 uses randomness only for the start: same start, largest difference 0.00e+00; different start, largest
    difference 1.80
one eta-0 jump from t = 700 to 300 with the true x_0 lands on sqrt(abar_300) x_0 + sqrt(1 - abar_300) eps, the same eps:
    largest difference 4.44e-16
Actual output from this lesson's run.
```

What it means

With $\eta = 1$ and a stride of one step, DDIM's fresh-noise variance equals Day 140's β-tilde at every t checked (e.g. 0.0100313554 at t = 500), and the two samplers, given the same guesser and the same random numbers, agree to 3.58e-14: DDIM contains Day 140's sampler as a special case. With $\eta = 0$ the only randomness is the starting noise (same start, difference 0.00e+00): one starting noise always gives the same digit.

The last line is why jumping is allowed. If the x_0 guess were perfect, one η-0 jump from t = 700 to 300 lands exactly where noising x_0 to step 300 with the same noise would (difference 4.44e-16). All of DDIM's error comes from the guess being imperfect, and longer jumps lean on fewer guesses.

WHY CHOOSE THIS?

The same trained model, no retraining, and any number of steps from 1,000 down to 1.

WHY NOT THIS? (TRADEOFFS)

Checked here on a made-up guesser; whether fewer steps still give good digits is a separate question (section 5).

2. Three models, one scorer

DEFINITION — start here, no prior knowledge assumed

Each model is trained on the same 60,000 training digits and asked for 2,000 samples from the same random seed. Day 138's scorer compares each set with real digits: **IS** (are the samples confidently one digit each, and are all ten digits there?), **FID** (how far the samples' features sit from real digits' features; lower is better), **precision** (the share of samples that look like real digits), **recall** (the share of real digits the samples cover) and **nearest** (the pixel distance to the closest training digit).

REAL-WORLD SCENARIO

Three bakeries entered in one competition with the same judges and the same flour: the scores mean something only because everything except the recipe is held fixed.

SPECIFICATION

VAE	Day 129's: hidden layers of 256, 16 codes, 30 epochs; samples decoded from codes drawn from N(0, I)
GAN	Day 134's dense players: generator [64, 256, 784], critic [784, 256, 1]; AdamGAN lr 0.0002, b1 0.5; 30 epochs, scored at 1, 5, 10, 20, 30
Diffusion	Day 141's model (output read as x_0, output layer started at zero), 30,000 steps; sampled from the averaged copy (section 3) with Day 140's 1,000-step sampler
Scorer	Day 138's: a classifier trained on the training digits; reference real digits from the training set

IMPLEMENTATION

```
def demo_gan(X, sc):
    G = d126.init_net([64, 256, 784], 0); D = d126.init_net([784, 256, 1], 1)
    oG, oD = d134.AdamGAN(G, 2e-4, 0.5), d134.AdamGAN(D, 2e-4, 0.5); rng =
        np.random.RandomState(0)
    Z = np.random.RandomState(1).standard_normal((2000, 64)).astype(np.float32); log = []; took =
        0.0
    for ep in range(1, max(GAN_EPOCHS) + 1):
        t0 = time.perf_counter(); o = rng.permutation(60000)
        for s in range(0, 60000 - 127, 128):
            d134.gan_step(G, D, oG, oD, X[o[s:s + 128]], rng, 64)
        took += time.perf_counter() - t0
        if ep in GAN_EPOCHS:
            t0 = time.perf_counter(); F, _ = d134.generate(G, Z); ts = time.perf_counter() - t0
            F = F.astype(np.float32); log.append((ep, *sc(F)))
    return dict(name="GAN", F=F, train=took, sample=ts, calls=1, scores=sc(F), params=n_params(G,
        D), gen=n_params(G), log=log)
```

ACTUAL OUTPUT

```
GAN while training:
   epoch      IS      FID  precision  recall  nearest
       1   1.200  435.586      1.000   0.000     4.72
       5   4.606   68.793      0.653   0.158     5.80
      10   4.961   49.446      0.515   0.463     5.88
      20   5.085   50.378      0.467   0.614     6.17
      30   5.186   52.781      0.547   0.641     5.82
    model      IS      FID  precision  recall  nearest
      VAE   5.405   38.520      0.676   0.736     5.03
      GAN   5.186   52.781      0.547   0.641     5.82
 diffusion   5.880   27.778      0.780   0.757     4.48
Actual output from this lesson's run.
```

What it means

The diffusion model scored best on all four quality measures: FID 27.778, precision 0.780, recall 0.757, and IS 5.880. The VAE came second (FID 38.520), and the GAN last (52.781). The GAN's own history shows why: it improved fast (FID 435.586 after one epoch, 49.446 after ten) and then drifted slightly worse (52.781 at 30), with low precision (0.547) throughout: its digits are sharp but often malformed.

This ranking holds for these three models, as built here: dense networks of similar size, one run each. It is not a ranking of the families. Day 139's convolutional, class-conditioned GAN reached FID 20.2, better than every model here; architecture matters as much as the family.

WHY CHOOSE THIS?

One scorer, one seed, one machine: the differences come from the models, not the measuring.

WHY NOT THIS? (TRADEOFFS)

One run per model; FID moves by several points between runs (Day 138), so close results would need repeats.

3. Raw weights against an averaged copy

DEFINITION — start here, no prior knowledge assumed

Day 141's samples got worse after 10,000 steps while the loss kept falling. A standard fix is an **averaged copy** of the weights (an exponential moving average, EMA): after every training step, the copy moves 0.1% of the way towards the current weights. It is never trained itself; it smooths out the step-to-step jitter of training. Both copies are scored at four points.

REAL-WORLD SCENARIO

A ship's navigator who plots the average of the last hour's readings rather than the latest one: a single gust does not swing the course, but a real turn takes a while to show up.

MATHEMATICS

$$W_{avg} \leftarrow d \cdot W_{avg} + (1 - d) \cdot W, \quad d = 0.999, \text{ after every Adam step}$$

weight of the step k steps ago: $(1 - d)\, d^k$; half of the total weight is on the last $\ln 2 / (1 - d) \approx 693$ steps

IMPLEMENTATION

```
def ema_update(ema, net, decay=DECAY):
    for (We, be), (W, b) in zip(ema, net):
        We *= decay; We += (1 - decay) * W
        be *= decay; be += (1 - decay) * b
```

ACTUAL OUTPUT

```
the averaged copy: after every Adam step, W_avg = 0.999 W_avg + 0.001 W, so it is an average over roughly the last 1,000
    steps;
it is never trained itself. Day 141 saw the raw model's samples get worse after 10,000 steps while its loss kept falling
    step   weights      IS      FID  precision  recall  nearest  grey share
   5,000       raw   7.325   48.992      0.617   0.620     4.67      0.0485
   5,000  averaged   4.266   75.153      0.858   0.653     4.89      0.1192
  10,000       raw   6.916   36.856      0.739   0.513     4.57      0.0591
  10,000  averaged   5.660   32.219      0.799   0.699     4.54      0.0907
  20,000       raw   6.692   56.008      0.629   0.601     5.30      0.0679
  20,000  averaged   5.982   24.887      0.780   0.755     4.45      0.0855
  30,000       raw   6.809   63.350      0.652   0.535     5.24      0.0801
  30,000  averaged   5.880   27.778      0.780   0.757     4.48      0.0851
Actual output from this lesson's run.
```

What it means

From 10,000 steps on, the averaged copy was better every time, and it did not decline: FID 32.219, 24.887 and 27.778, against the raw weights' 36.856, 56.008 and 63.350. The raw numbers repeat Day 141's exactly, because training is the same; only the copy that is sampled differs.

At 5,000 steps the averaged copy was much worse (75.153 against 48.992), with very blurred samples (grey share 0.1192). A likely reason, not tested here: early in training the weights change fast, and an average over the last thousand steps mixes models that disagree, which blurs their guesses. An averaged copy helps once the weights are jittering around a good place, not while they are still moving towards it.

WHY CHOOSE THIS?

A few lines of code, no extra training, and the averaged model held its quality where the raw one fell back.

WHY NOT THIS? (TRADEOFFS)

A second full copy of the weights in memory, and it lags: early in training it was the worse model.

4. What the scores miss: blur

DEFINITION — start here, no prior knowledge assumed

FID compares features from a classifier; a classifier can recognise a blurred digit perfectly well. So a direct measure of blur is added: the **grey share**, the share of all pixels between 0.25 and 0.75 (on a 0–1 scale), neither background nor ink. Real digits have a few grey pixels at the edges of each stroke; a blurred digit has many more.

REAL-WORLD SCENARIO

A spell-checker that passes a letter with every word spelled right and the ink smudged: it checks what it was built to check.

IMPLEMENTATION

```
def grey_share(F, lo=0.25, hi=0.75):
    return float(((F > lo) & (F < hi)).mean())
```

ACTUAL OUTPUT

```
grey share: the share of all pixels between 0.25 and 0.75, neither background nor ink; real digits have a little, at
the edges of strokes; a blurred digit has much more. Ink: the average pixel.
            images  grey share     ink      FID
  real test digits      0.0500   0.132    2.996
               VAE      0.0890   0.116   38.520
               GAN      0.0313   0.121   52.781
         diffusion      0.0851   0.122   27.778
Actual output from this lesson's run.
```

What it means

Real digits have a grey share of 0.0500. The VAE's samples have 0.0890, almost twice as much, yet its FID (38.520) beat the GAN's (52.781), whose samples are the sharpest of all (0.0313, below real). The figure shows the trade plainly: the VAE's digits are soft but well formed, the GAN's are crisp but often broken. The diffusion model's samples are also blurred (0.0851), less than the VAE's.

Neither number alone is the right one. FID ranks the VAE above the GAN; the grey share ranks them the other way. A real evaluation reports several measures and looks at the samples, as the figure here does.

WHY CHOOSE THIS?

One line of code that measures exactly what FID ignores, calibrated by real digits.

WHY NOT THIS? (TRADEOFFS)

It only measures blur, and only for images that should be black on white; a sharp but wrong digit scores as well as a real one.

5. Fewer steps: DDIM from 1,000 steps down to 5

DEFINITION — start here, no prior knowledge assumed

The averaged diffusion model, trained once, is sampled with DDIM at seven step counts and both settings of η, always from the same starting noise. Every step is one network call, so the time should fall roughly in proportion to the steps.

REAL-WORLD SCENARIO

Sketching a portrait in 1,000 small strokes or in 10 bold ones: the bold version is done in seconds, but details that need many small corrections go missing.

SPECIFICATION

Model	the averaged copy after 30,000 steps (sections 2–3)
Steps	1,000, 250, 100, 50, 20, 10, 5, evenly spaced from t = 1,000 to 0
η	0 (no fresh noise after the start) and 1 (fresh noise as in Day 140's sampler)
Samples	2,000 each, from seed 1; time measured around the sampler only

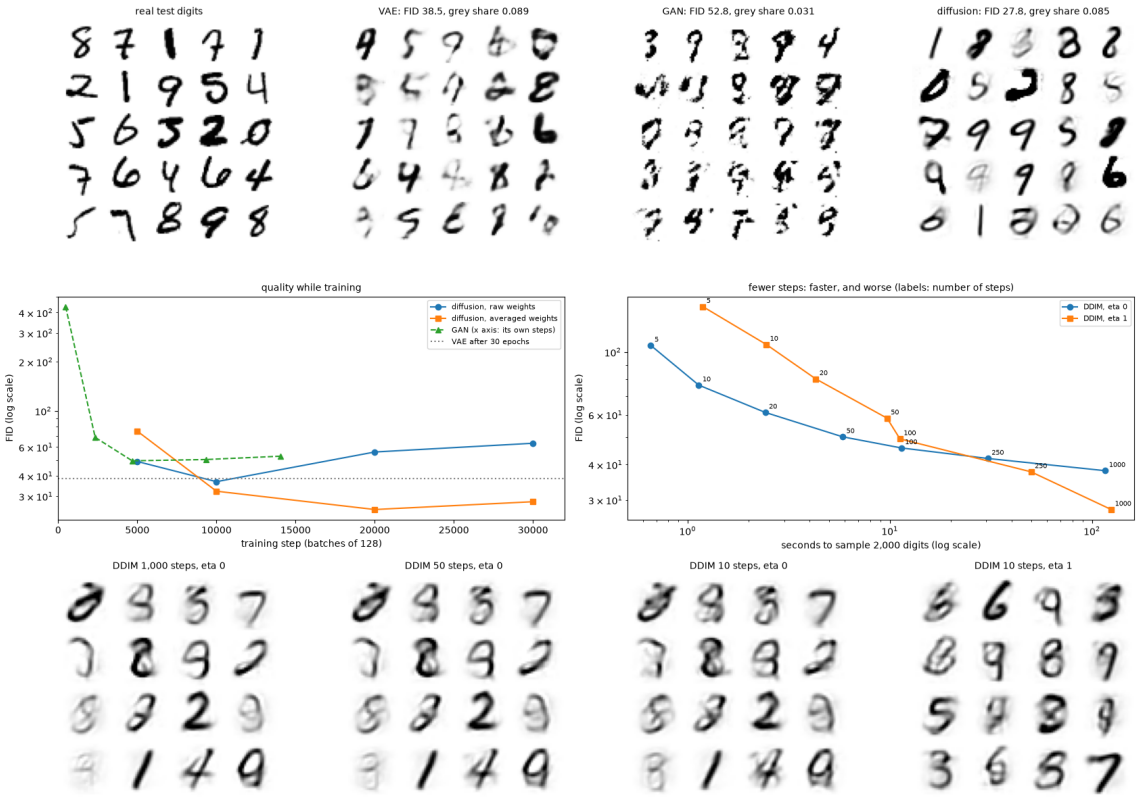
ACTUAL OUTPUT

```
2,000 samples each from the same starting noise (seed 1); eta 0: no fresh noise after the start; eta 1: fresh noise as in
    DDPM
 steps  eta      IS      FID  precision  recall  grey share    time
  1000    0   5.770   38.086      0.841   0.683      0.1031  116.4s
   250    0   5.760   42.073      0.862   0.669      0.1087   30.6s
   100    0   5.727   45.837      0.869   0.652      0.1133   11.4s
    50    0   5.663   50.174      0.874   0.634      0.1178    5.8s
    20    0   5.389   61.201      0.884   0.551      0.1264    2.4s
    10    0   4.989   76.389      0.903   0.459      0.1361    1.1s
     5    0   4.248  105.503      0.930   0.309      0.1525    0.7s
  1000    1   5.880   27.778      0.780   0.757      0.0851  124.5s
   250    1   5.617   37.743      0.815   0.705      0.0952   50.0s
   100    1   5.468   49.334      0.851   0.663      0.1035   11.2s
    50    1   5.315   58.388      0.866   0.604      0.1109    9.7s
    20    1   4.804   80.382      0.898   0.460      0.1245    4.3s
    10    1   4.238  106.351      0.931   0.350      0.1384    2.4s
     5    1   3.431  144.729      0.961   0.165      0.1563    1.2s
Actual output from this lesson's run.
```

What it means

Quality fell smoothly as the steps were cut. With η = 1, FID went from 27.778 at 1,000 steps to 49.334 at 100 and 144.729 at 5; recall fell with it (0.757 to 0.165) while precision rose (0.961 at 5 steps): fewer steps gave fewer kinds of digit, each safer. The grey share rose at every cut: fewer steps, more blur.

The two settings of η cross. At 1,000 steps fresh noise helped (FID 27.778 against 38.086 without); at 100 steps and below, no fresh noise was better (45.837 against 49.334 at 100, 61.201 against 80.382 at 20). A likely reason, not tested here: fresh noise corrects earlier mistakes, which needs many small steps to work; with few large steps there is no time to recover from the noise added. This matches why DDIM, with η = 0, is the usual choice for fast sampling.



Top: 25 samples each: real test digits, VAE, GAN, diffusion. Middle: FID while training (log scale); FID against sampling time for DDIM. Bottom: the same 16 starting noises sampled with η = 0 in 1,000, 50 and 10 steps (the same digits, more blurred), and with η = 1 in 10 steps.

WHY CHOOSE THIS?

A slider between speed and quality on one trained model; with η = 0, one noise always gives one image, which makes results repeatable.

WHY NOT THIS? (TRADEOFFS)

Each halving of the steps cost quality here; 20 steps was not free (FID 61.201 against 27.778).

6. The tradeoffs in one table

DEFINITION — start here, no prior knowledge assumed

Every cost and every score from sections 2–5 in one table. **Train** is wall-clock training time (for diffusion, the time spent sampling at the four checkpoints is taken out; the averaging updates, and the few seconds of scoring, stay in); **sample 2,000** is the time to generate 2,000 digits; **net calls** is how many times the network runs per sample; **numbers** counts every trained weight, and **sampler** only those needed to generate.

REAL-WORLD SCENARIO

A buyer's guide that puts price, speed and quality side by side: the best choice depends on which column matters to the buyer.

ACTUAL OUTPUT

```
6. THE TRADEOFFS IN ONE TABLE -- all on this machine, 4 threads, 2,000 samples
======================================================================
                 model   train  sample 2,000  net calls      FID  recall    grey   numbers   sampler
                   VAE    100s         0.02s          1   38.520   0.736  0.0890   415,024   205,840
                   GAN    125s         0.01s          1   52.781   0.641  0.0313   419,345   218,128
diffusion, 1,000 steps   1244s        97.06s      1,000   27.778   0.757  0.0851 1,083,152 1,083,152
    diffusion, DDIM 20   1244s         2.42s         20   61.201                1,083,152 1,083,152
'numbers': every weight trained (VAE: encoder + decoder; GAN: both players); 'sampler': the part needed to generate
Day 139's conditional DCGAN, for reference (not rerun): 6,387 s for 10 epochs, FID 20.200, recall 0.573
```

What it means

Training: the VAE took 100s, the GAN 125s, the diffusion model 1244s, about ten times as long, for its 30,000 steps of a network 2.6 times the size of the VAE's encoder and decoder together (1,083,152 numbers against 415,024). Sampling is where the gap is enormous: 0.02s for 2,000 VAE digits and 0.01s for the GAN, one network call each, against 97.06s for diffusion's 1,000 calls. DDIM with 20 steps brought it to 2.42s, still more than a hundred times as long as the VAE or the GAN, and its FID (61.201) was then worse than the VAE's.

The times carry real noise. The first run of this script was interrupted by the laptop going into standby (Windows' power log shows standby from 1:00 to 2:37 while it ran) and timed diffusion training at 8,174 s; it was run again, plugged in and kept awake, and every score came out identical. Even in this run the DDIM times are uneven: 250 steps with $\eta = 1$ took 50.0s and 100 steps 11.2s, where the steps alone predict 2.5 times as long. Read the times for their orders of magnitude, not their decimals.

WHY CHOOSE THIS?

Real measured costs on one machine, beside the quality they bought.

WHY NOT THIS? (TRADEOFFS)

Times depend on the machine and on code that is not tuned for speed; the ratios mean more than the seconds.

Interview preparation — technical questions

Q: How does DDIM sample faster from the same trained model?

A: It jumps from step t to an earlier step s using the x_0 guess and the noise it implies; here 20 steps took 2.42s instead of 97.06s.

Q: What is an EMA of the weights, and did it help?

A: A copy that moves 0.1% towards the current weights each step. Here FID 27.778 against 63.350 for the raw weights at 30,000 steps; worse at 5,000 (75.153 against 48.992).

Q: Why can FID rank a blurry model above a sharp one?

A: It compares classifier features, and the classifier recognises blurred digits. Here the VAE (grey share 0.0890) beat the GAN (0.0313) on FID.

Q: When is $\eta = 0$ better than $\eta = 1$?

A: With few steps: at 20 steps FID 61.201 against 80.382; with all 1,000 steps $\eta = 1$ was better (27.778 against 38.086).

Q: How would you compare generative models fairly?

A: Same data, same scorer, same seed, same machine; several measures including one for blur; and look at the samples.

Interview preparation — theoretical questions

Q: Why does a diffusion model cost so much more to sample than a GAN?

A: A GAN or VAE runs its network once per sample; a diffusion model runs it once per step, 1,000 times with Day 140's sampler.

Q: Why does a VAE blur?

A: Its decoder is trained on a squared-error-like loss; when several images fit one code, the best guess is their average, and an average of sharp digits is a blurred one.

Q: Why are GAN samples sharp but sometimes broken?

A: The critic punishes blur that real digits never have, but it does not require every stroke to be in place; nothing pulls each sample towards a real image.

Q: Does DDIM need a new model?

A: No: it uses the same trained network; with $\eta = 1$ and all 1,000 steps it is exactly Day 140's sampler (difference 3.58e-14 here).

Q: Which family is best?

A: None in general. Here diffusion had the best scores and the highest costs, and Day 139's convolutional GAN beat all three: architecture matters as much as family.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Measured on one machine and one scorer, the trade is real: the diffusion model gave the best samples and cost the most, in training and above all in sampling. DDIM narrows the sampling gap from the same trained model, at a price in quality at every cut, and an averaged copy of the weights fixed Day 141's late decline. No single score told the whole story: FID missed the VAE's blur, which a count of grey pixels and a look at the samples both showed.

- FID: diffusion 27.778, VAE 38.520, GAN 52.781 (real test digits 2.996).
- Averaged copy against raw weights after 30,000 steps: FID 27.778 against 63.350.
- Grey share: real 0.0500, VAE 0.0890, diffusion 0.0851, GAN 0.0313.
- DDIM with $\eta = 0$: 1,000 steps FID 38.086, 50 steps 50.174, 5 steps 105.503.
- Sampling 2,000 digits: VAE 0.02s, GAN 0.01s, diffusion 97.06s (DDIM 20 steps: 2.42s).

Suggested watch order, if starting from zero

- 1. Generative Models 101: VAEs, GANs, Autoregressive & Diffusion Models Explained (CorestratAI) — an overview of the families side by side, before the numbers.
- 2. Denoising Diffusion Implicit Models (DDIM) Explained (ExplainingAI) — DDIM's derivation, which section 1 checks by hand.

Day 143 preview

- **CLIP-style joint embeddings**: an image encoder and a text encoder trained so that a digit and its caption land close together, then used to recognise digits from words alone.